

Experiment No: 4

Title: Use function types, scope, and closures; apply try-catch; build a palindrome checker.

Software/Tools Required:

JavaScript, HTML, CSS and Code Editor.

Theory:

Functions, Scope, Closures, and Exception Handling in JavaScript

JavaScript functions are reusable blocks of code designed to perform specific tasks. They help organize code, improve readability, and reduce repetition. JavaScript supports different types of functions and provides concepts such as scope and closures for efficient data management. Exception handling using try-catch helps manage runtime errors and ensures smooth program execution.

1. Function Types in JavaScript

A function is a set of statements that performs a particular task when called.

a) Function Declaration

A named function defined using the function keyword.

b) Function Expression

A function assigned to a variable.

c) Arrow Function

A concise syntax introduced in ES6.

2. Scope in JavaScript

Scope determines the accessibility of variables within different parts of a program.

a) Global Scope

Variables declared outside any function are accessible throughout the program.

b) Local Scope

Variables declared inside a function can only be accessed within that function.

c) Block Scope

Variables declared using `let` and `const` are limited to the block in which they are defined.

3. Closures in JavaScript

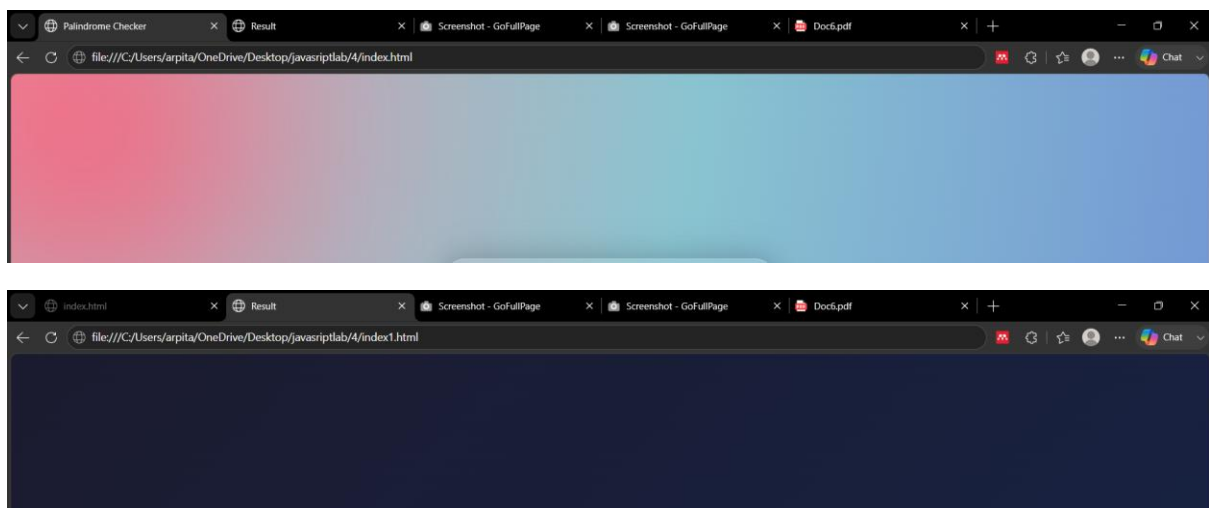
A closure is a feature in which an inner function retains access to variables of its outer function even after the outer function has finished execution. Closures help in data encapsulation and maintaining private variables.

4. Exception Handling Using `try-catch`

Errors may occur during program execution due to invalid input or unexpected conditions. JavaScript provides `try-catch` blocks to handle such errors gracefully.

- **try** – Contains code that may generate an error.
- **catch** – Executes when an error occurs.
- **finally** (optional) – Executes regardless of whether an error occurs.

Output



Case Study 1: Palindrome Checker

Case Study Program Code

Problem Statement

Design a **Palindrome Checker** that accepts a word from the user and determines whether it is a palindrome or not. The program should validate the input, handle invalid entries using **try-catch**, and display the result dynamically.

Implementation

The webpage consists of an HTML form where the user enters a word. JavaScript uses **functions**, **closures**, and **try-catch** for input validation and error handling. The entered word is reversed and compared with the original word to determine whether it is a palindrome. The result is displayed dynamically on the webpage.

Case Study 2: Vehicle Registration Validation

Case Study Program Code

Problem Statement

Design a **Vehicle Registration Validation System** that accepts a vehicle registration number and verifies whether it follows the standard registration format. The program should display a success message for valid input and appropriate error messages for invalid input.

Implementation

The webpage consists of an HTML form where the user enters a vehicle registration number. JavaScript uses **functions**, **closures**, and **try-catch** for validation and exception handling. A **regular expression** is used to verify the registration number format, and the result is displayed dynamically on the webpage.

Conclusion:

Functions enable code reusability and modular programming in JavaScript. Scope controls variable accessibility, while closures allow functions to retain access to their surrounding environment. Exception handling using try-catch improves program reliability by managing runtime errors. These concepts can be effectively demonstrated through the development of a palindrome checker that validates and processes user input efficiently.

